BAB 2 : MODEL-MODEL DAN PROSES PENGEMBANGAN PERANGKAT LUNAK

Tujuan Khusus:

Pada bab ini mahasiswa diharapkan mampu :

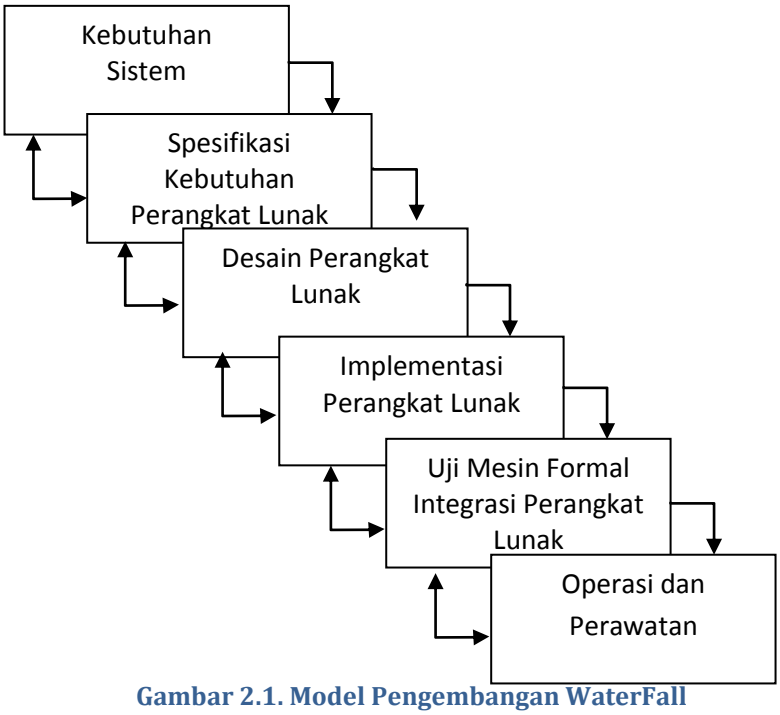
- *Memahami Model-model dan proses pengembangan perangkat lunak*
- *Mampu Memilih model pengembangan yang sesuai untuk proyek pengembangan perangkat lunak.*
- *Mampu membedakan fungsi dan penggunaan masing-masing model sesuai dengan proyek pengembangan perangkat lunak yang ada di dunia nyata.*
- *Memahami tingkatan pengembangan perangkat lunak dan kematangan perangkat lunak serta kualitas menggunakan standar kualitas seperti SEI-CMM, Malcolm Bridge dan ISO 9000 .*

2.1 PENDAHULUAN

Pada bab dua ini akan dibahas secara umum model proses yang digunakan dalam komunitas pengembang perangkat lunak. Model proses pengembangan perangkat lunak pada bab ini dimulai dengan model air terjun. Kemudian dilanjutkan dengan pendekatan prototipe, spiral, iteratif dan pendekatan proses berorientasi objek. Proses serupa dilakuakn untuk peningkatan proses pengembangan seperti CleanRoom dan pencegahan kerusakan. Pembahasan pada bab ini akan dibahas dalam kerangka kematangan proses, termasuk *Capability Maturity Model* dari *Software Engineering Institute* dan pendekatan dalam *Software Productivity Research* serta dua bentuk standar kualitas yakni Malcom Baldrige dan ISO 9000

2.2 MODEL PENGEMBANGAN *WATERFALL*

Seperti telah diceritakan pada sejarah perkembangan rekayasa perangkat lunak pada bab sebelumnya, pada era tahun 1960-an dan 1970-an proyek pengembangan perangkat lunak mengalami krisis dan sangat memakan biaya serta waktu pengembangan yang lama. Kemunculan model pengembangan *Waterfall* adalah untuk membantu mengatasi kerumitan yang terjadi akibat proyek-proyek pengembangan perangkat lunak. Penyajian sederhana dari model pengembangan *waterfall* dapat dilihat pada gambar 2.1 dibawah ini :



Gambar 2.1. Model Pengembangan WaterFall

Model ini membagi pengembangan yang kompleks menjadi beberapa langkah sederhana yang membawa menuju produk akhir yang siap pakai. Untuk memastikan bahwa sistem dapat dijalankan makan setiap langkah membutuhkan validasi, masukkan dan kriteria.

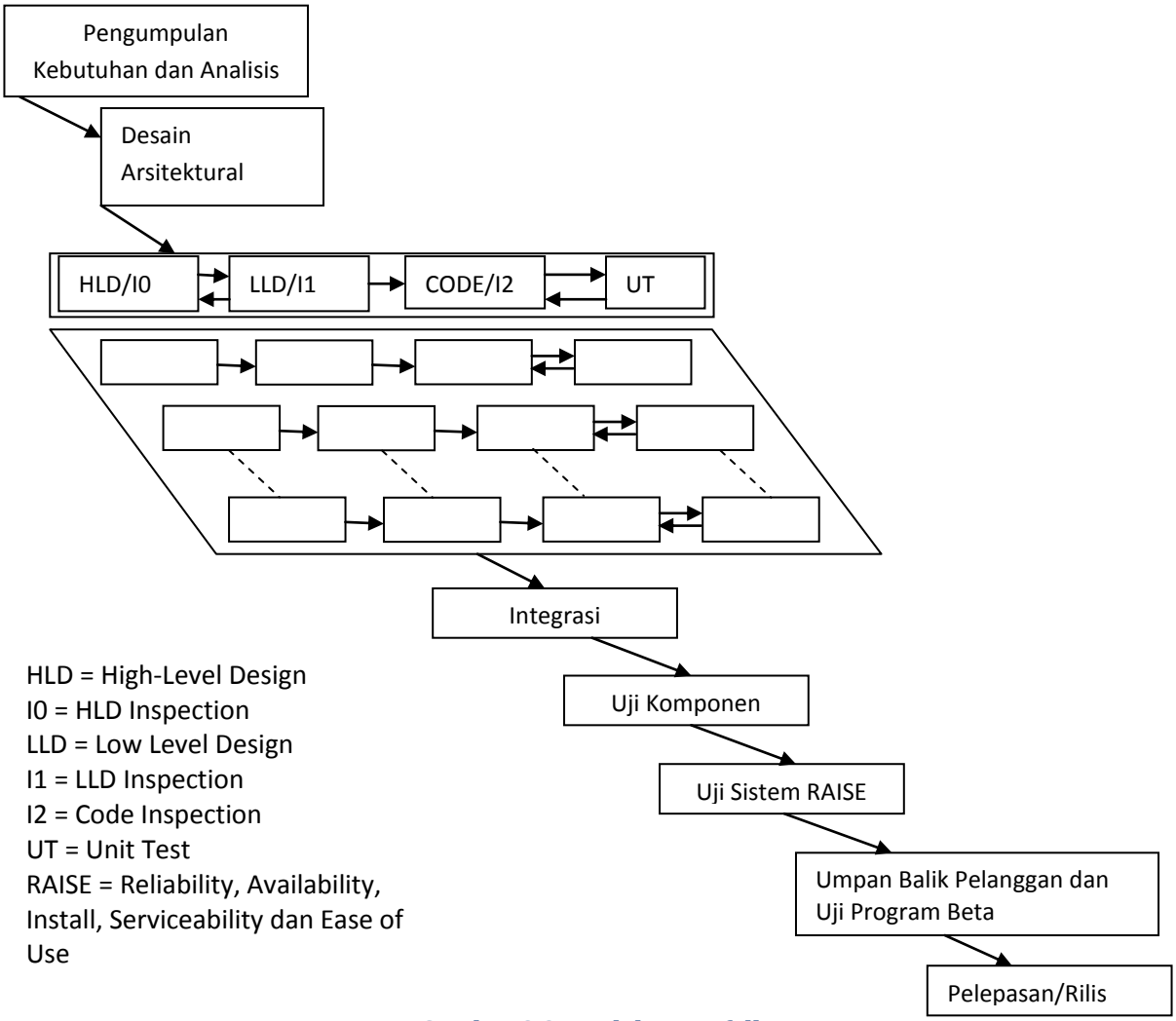
Karakteristik dari model waterfall ini adalah paradigma *entry-task-validation* (ETV). Paradigma ini memungkinkan pelacakan yang lebih akurat dengan perkembangan proyek dan dapat mengidentifikasi dengan jelas apabila terjadi kesalahan.

Keuntungan pendekatan model ini adalah membuat perangkat yang lebih besar menjadi mudah diatur dan selesai tepat pada waktunya tanpa biaya yang berlebihan. Banyak pengembangan besar yang telah lama berdiri yang terlibat dalam pengembangan proses ini telah menggunakannya. Tahapan pengembangan juga dapat dibagi menjadi desain tingkat tinggi dan desain tingkat rendah (terperinci); tahapan implementasi biasa disebut *code* dan *Debug;* serta tahapan pengujian yang mencakup uji tingkat komponen, uji tingkat produk dan uji tingkat sistem.

Pengembangan model air terjun untuk proyek pengembangan yang lebih besar dapat dilihat pada gambar 2.2. Pada gambar tersebut diperlihatkan bahwa tahapan kebutuhan diikuti tahap perancangan arsitektur. Ketika arsitektur sistem dan perancangan sudah tersedia, masing-masing fungsi perancangan dan pengembangan akan dikerjakan. Fungsi yang dimaksud terdiri dari *High Level Design* (HLD), *Low Level Design* (LLD), *Code Development* dan *Unit Testing*(UT).

Pada gambar 2.2. juga tampak bahwa tahap pengembangan kode dan pengujian unit juga dijalankan secara bersamaan. Karena UT adalah bagian dari tahap implementasi, kurang masuk akal bila proses ini dipisahkan menjadi bagian tersendiri.

Ketika pengkodean dan pengujian unit telah selesai. Tahapan selanjutnya adalah integrasi, uji komponen, uji sistem dan program awal dapat dimulai untuk pelanggan. Tahap yang terakhir adalah pelepasan perangkat lunak yang telah selesai dikerjakan pelanggan.



Gambar 2.2. Model Waterfall

2.2.1 HIGH LEVEL DESIGN

High Level Design (Desain Tingkat tinggi) menyediakan sebuah solusi, platform, sistem, produk, layanan ataupun proses seperti sebuah gambaran penting dalam pengembangan multi-proyek untuk memastikan bahwa masing-masing komponen desain pendukung sesuai dengan kenyataannya.

Desain tingkat tinggi harus menjelaskan secara jelas seluruh platform, sistem, produk, layanan dan proses dimana ia bergantung dan memasukkan perubahan penting yang dibutuhkan dan harus dilakukan.

Dokumen desain tingkat tinggi akan selalu memasukan diagram arsitektur yang menggambarkan komponen-komponen, antar-muka dan jaringan yang perlu lebih jauh dikembangkan di spesifikkan. Dokumen juga menggambarkan atau sebaliknya menunjukkan arus kerja dan (atau) arus data antara sistem-sistem komponen.

Oleh karenanya, *High Level Design* dikatakan sebagai proses untuk menentukan sisi internal dan eksternal dari perspektif sebuah komponen. Tujuan utamanya adalah sebagai berikut :

1. Mengembangkan fungsi eksternal dan antarmuka, yaitu :
    a. Antarmuka pengguna eksternal
    b. Antarmuka program aplikasi
    c. Antarmuka program sistem
    d. Antarmuka antarkomponen dan struktur data
2. Desain sruktur komponen internal yang mencakup antarmuka antarkomponen dan struktur data
3. Memastikan bahwa semua kebutuhan fungsi telah terpenuhi
4. Memastikan bahwa semua komponen sesuai dengan struktur produk dan sistem
5. Memastikan bahwa desain komponen diselesaikan dengan sempurna
6. Memastikan fungsi eksternal dapat dilakukan

2.2.2 LOW LEVEL DESIGN

Low Level Design (Desain Level Tingkat Rendah) adalah proses menjadikan HLD menjadi desain yang lebih terperinci menggunakan sudut pandang per bagian. LLD mendefinisikan logika aktual untuk masing-masing komponen sistem. Diagram kelas dengan seluruh metodenya dan relasi antar kelas termasuk LLD. Tujuan dari Desain Level Tingkat Rendah adalah :

1. Penyelesaian desain komponen dan bagiannya (*modules, macros, includes*) di dalam sistem atau produk
2. Menyelesaikan seluruh rencana uji komponen, dan
3. Memberikan umpan balik tentang HLD dan verifikasi perubahan yang dilakukan HLD

LLD menjelaskan masing-masing dan setiap modul dengan metode elaborasi sehinga programmer dapat secara langsung meng-kode perogram. Ada paling tidak satu dokumen untuk masing-masing modul dan boleh jadi ada lebih banyak untuk sebuah modul. LLD akan berisikan antara lain :

- Logika fungsional yang dari modul dalam psudokode
- Tabel Database dengan semua elemen-elemen termatuk tipe dan ukurannya
- Semua antarmuka secara terperinco dengan referensi API yang lengkap (baik untuk request maupun respon)
- Semua isu ketergantungan
- Daftar pesan kesalahan
- Keseluruhan input dan output lengkap untuk sebuah modul

2.2.3 TAHAP KODING (PENGKODEAN)

Tahapan pengkodean (koding) menghasilkan perubahan fungsi LLD menjadi kode bahasa pemrograman yang dapat diuji hasilnya. Tugasnya adalah sebagai berikut :

1. Membuat kode untuk setiap bagian (modules, macros, includes, message dan seterusnya)
2. Melakukan pengujian terhadap pengkodean yang telah dilakukan
3. Melakukan verifikasi terhadap perubahan HLD dan LLD

2.2.4 UNIT TES

Unit Tes adalah pengujian pertama dari modul yang dapat dieksekusi (*executable*). Tujuannya adalah sebagai berikut :

- Memverifikasi kode untuk desain tingkat tinggi(HLD) dan desain tingkat rendah(LLD)
- Mencoba seluruh kode, baik kode baru maupun kode lama untuk memastikan fungsinya dilakukan ke seluruh sistem, pembenaran logika dan verifikasi jalur data.
- Melihat kembali seluruh pesan kesalahan (*error*), kode kembali dan reaksi
- Memberikan umpan balik mengenai kode, HLD dan LLD

Tingkat unit tes ini uji fungsi adalah verifikasi batas, antarmuka internal, jalur logika dan data di dalam model, makro atau seluruh unit yang bisa dijalankan. Unit tes dilakukan pada kode yang tidak terintegrasi dan membutuhkan kode tersendiri untuk menciptakan lingkungan yang sesuai.

2.2.5 UJI KOMPONEN

Uji komponen mengevaluasi gabungan dari bagian perangkat lunak seluruhnya diintegrasikan ke dalam sistem. Tujuannya adalah :

a. Uji antarmuka dengan pengguna eksternal terhadap dokumentasi desain komponen
b. Uji antarmuka program aplikasi dengan dokumentasi desain komponen
c. Uji fungsi terhadap dokumentasi desain komponen
d. Uji antarmuka antarkomponen(tingkat modul) terhadap dokumentasi desain komponen

e. Uji pemulihan kerusakan dan pesan terhadap dokumentasi desain desain komponen
f. Verifikasi komponen sudah selesai dan bisa diterima dalam batasan kualitas
g. Uji jalur gabungan(*multitasking*) dan sumber gabungan (file, lock,queues dst) terhadap dokumentasi desain komponen, dan
h. Uji fungsi yang tidak diubah terhadap dokumentasi desain komponen.

2.2.6 UJI TINGKAT SISTEM

Uji sistem terbagi menjadi beberapa bagian antara lain :

1. Uji Sistem
   a. Memastikan bahwa produk berjalan sebagaiman mestinya ketika dijalankan secara periodik maupun secara terus-menerus dan
   b. Memverifikasi keseluruhan kestabilan program ketika seluruh pengembangan telah selesai dikerjakan untuk semua produk.
2. Uji Regresi Sistem
   a. Memverifikasi paket program akhir yang telah siap dikirimkan kepada pelanggan dan
   b. Memastikan fungsi asli tetap berjalan setelah seluruh fungsi ditambahkan ke dalam sistem.
3. Uji Pengukuran kinerja sistem
   a. Memvalidasi kinerja sistem
   b. Memverifikasi spesifikasi kinerja
   c. Menyediakan kinerja informasi kepada pihak pemasaran dan
   d. Membuat pedoman pengukuran untuk peluncuran selanjutnya
4. Uji Kegunaan
   a. Uji kegunaan mencakup verifikasi sistem yang memiliki karakteristik yang dibutuhkan pengguna dan lingkungan pengguna.

Uji sistem dilakukan setelah uji komponen dan sebelum uji sistem regresi. Uji daya kerja sistem biasanya dimulai setelah uji sistem dimulai dan terus berlangsung pada semua tahap ujicoba. Uji kegunaan terjadi selama proses pengembangan.

2.2.7 PROGRAM AWAL UNTUK NASABAH

Program Awal Nasabah meliputi beberapa pengujian dari struktur yang didukung dan pemverifikasian kesiapannya, yaitu :

1. Struktur layanan nasabah
2. Mendukung pengembangan perbaikan
3. Mendukung pelanggan elektronik
4. Mendukung pemasaran
5. Pemesanan, produksi dan distribusi

Selain hal tersebut diatas, keuntungan lain dari kepemilikan program sistem di lingkungan nasabah dengan program awal nasabah adalah terbukanya kesempatan untuk mengumpulkan

umpan balik dari pengguna agar pengembang dapat menganalisis dan memperbaikinya pada program selanjutnya. Dapat dijelaskan dengan terperinci sebagai berikut :

1. Umpan balik produk
    a. Fungsi yang ditawarkan
    b. Kemudahan dalam penggunaan
    c. Kualitas dari dokumentasi secara on-line
2. Kemudahan pemasangan perangkat keras dan perangkat lunak
3. Produk yang dapat dipercaya
4. Kinerja
5. Konektivitas sistem dan
6. Kepercayaan konsumen

Hal tersebut diatas menggambarkan suatu model *waterfall* untuk pendekatan disiplin pengembangan perangkat lunak. Hal-hal tersebut memang dikhususkan untuk pengembangan sistem yang berkarakter kompleks dan saling ketergantungan tinggi. Meskipun diekspresikan sebagai jeram *waterfall,* paralelisme dan pengulangan pada tahap proses sering muncul pada saat implementasi yang sebenarnya. Proses harus berbasis entitas bukan berbasis langkah demi langkah. Jika tidak suatu proses bisa terlalu kaku, tidak efektif dan efisien.

2.3 MODEL PROTOTYPING

Model Prototyping adalah sebuah metode pengembangan sistem dimana sebuah prototipe (taksiran sistem atau produk final) dibangun, diuji coba dan dikerjakan kembali seperlunya sampai sebuah prototipe yang dapat diterima benar-benar dapat dicapai dari sistem atay produk lengkap yang dapat dikembangkan saat ini. Model ini dapat bekerja dengan sangat baik dalam skenario dimana tidak semua kebutuhan proyek diketahui pada awalnya secara terperinci. Merupakan suatu proses *trial-and-error* yang dapat terjadi antara pengembang dan pengguna.

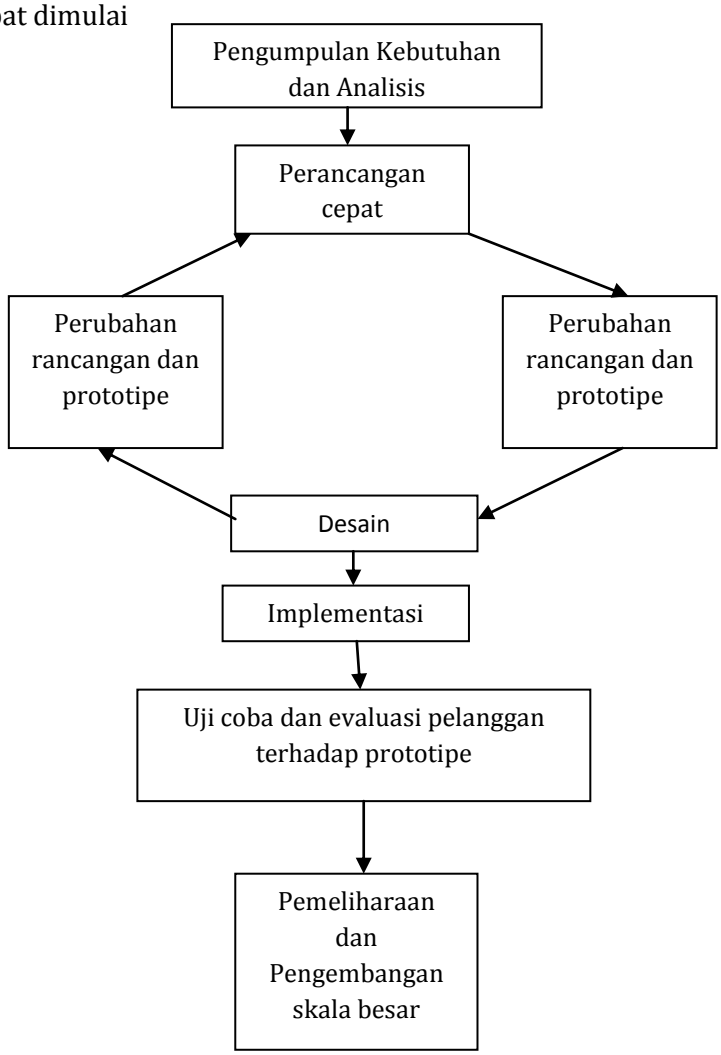
Ada beberapa tahapan dalam model prototyping :

1. Kebutuhan baru sistem didefinisikan dengan terperinci. Hal ini dapat melibatkan aktivitas wawancara terhadap sejumlah pengguna dari seluruh departemen atau aspek-aspek dari sistem yang ada saat ini.
2. Desain awal diciptakan untuk suatu sistem baru.
3. Sebuah prototipe pertama dari sistem yang baru dibangun dari desain awal. Hal ini selalu merupakan sebuah sistem *scaled-down,* dan menghadirkan sebuah perkiraan dari karakteristik produk akhir.
4. Pengguna sepenuhnya mengevaluasi prototipe pertama, memperhatikan kekuatan dan kelemahannya, hal-hal yang perlu ditambahkan dan yang harus dihilangkan. Pengembangan mengumpulkan dan menganalisis komentar yang berasal dari pengguna.
5. Prototipe pertama dimodifikasi didasarkan kepada komentar yang diberikan oleh pengguna, dan sebuah prototypr kedua dari sistem yang baru dibangun.
6. Prototipe kedua dievaluasi dengan cara yang sama dengan prototipe pertama.
7. Tahapan sebelumnya diulangi kembali seperlunya, sehingga pengguna puas bahwa prototipe menghadirkan hasil akhir yang diharapkan.
8. Sistem akhir dikonstruksi didasarkan pada prototipe akhir.

9. Sistem akhir sepenuhnya dievaluasi dan diuji coba. Pemeliharaan rutin lakukan secara terus menerus untuk mencegah kegagalan dalam skala besar dan untuk meminimumkan waktu penghentian

Sebuah prototipe adalah bagian dari produk yang mengekspresikan logika maupun fisik antarmuka eskternal yang ditampilkan. Konsumen potensial menggunakan prototipe dan menyediakan masukan untuk tim pengembang sebelum pengembangan skala besar dimulai. Melihat dan mempercayai menjadi hal yang diharapkan untuk dicapai dalam prototipe. Dengan menggunakan pendekatan ini, konsumen dan tim pengembang dapat mengklarifikasi kebutuhan dan interpretasi mereka. Gambar 2.3 menunjukkan pendekatan prototipe pada umumnya dan melibatkan beberapa langkah berikut :

1. Mengumpulkan dan menganalisis kebutuhan
2. Melakukan perancangan cepat
3. Membuat sebuah prototipe
4. Evaluasi dilakukan oleh pelanggan terhadap prototipe
5. Perubahan rancangan dan prototipe
6. Apabila pelanggan tidak puas dengan prototipe yang telah dibuat, ulang langkah 5
7. Apabila pelanggan puas terhadap prototipe yang telah dibuat, pengembangan produk skala besar dapat dimulai



Gambar 2.3. Pendekatan Model Prototipe

Pendekatan ini memiliki beberapa keuntungan :

1. Pemodelan membutuhkan partisipasi aktif dari end-user. Hal ini akan meningkatkan sikap dan dukungan pengguna untuk pengerjaan proyek. Sikap moral pengguna akan meningkat karena system berhubungan nyata dengan mereka.
2. Perubahan dan iterasi merupakan konsekuensi alami dari pengembangan system-sehingga end user memiliki keinginan untuk merubah pola pikirnya. Prototyping lebih baik menempatkan situasi alamiah ini karena mengasumsikan perubahan model melalui iterasi kedalam system yang dibutuhkan.
3. Prototyping mematahkan folosofi "end user tidak mengetahui secara detail apa yang dibutuhkan sampai mereka melihat implementasinya"
4. Prototyping adalah model aktif, tidak pasif, sehingga end user dapat melihat, merasakan, dan mengalaminya.
5. Kesalahan yang terjadi dalam prototyping dapat dideteksi lebih dini
6. Prototyping dapat meningkatkan kreatifitas karena membolehkan adanya feedback dari end user. Hal ini akan memberikan solusi yang lebih baik.
7. Prototyping mempercepat beberapa fase hidup dari programmer.

McLeod dan Schell (2001) mengemukakan bahwa alasan-alasan pemakai maupun spesialis informasi menyukai model prototipe adalah:

1. Komunikasi antara analis sistem dan pemakai membaik;
2. Analis dapat bekerja dengan lebih baik dalam menemukan kebutuhan pemakai;
3. Pemakai berperan lebih aktif dalam pengembangan sistem;
4. Spesialis informasi dan pemakai menghabiskan lebih sedikit waktu dan usaha dalam mengembangkan sistem;
5. Implementasi menjadi lebih mudah karena pemakai mengetahui sistem yang diharapkan.

Tetapi, terdapat beberapa kelemahan dari prototyping, kelemahan tersebut antara lain :

1. Prototyping memungkinkan terjadinya pengembalian terhadap kode, implementasi, dan perbaikan siklus hidup yang dugunakan untuk mendominasi sistem informasi.
2. Prototyping tidak menolak kebutuhan dari fase analisis sistem. Prototipe hanya dapat memecahkan masalah yang salah dan memberi kesempatan sebagai sistem pengembangan konvensional.
3. Perancangan issu numerik tidak dialamaykan oleh prototyping. Isu tersebut dapat dilupakan jika pengguna tidak berhati-hati.
4. Prototyping dapat mengurangi kreatifitas perancangan.

Prototyping terkadang dapat memberikan performansi yang lambat, membantu mendapatkan kebutuhan detil lebih baik namun demikian Prototipe juga menimbulkan masalah:

1. Dalam membuat prototipe banyak hal yang diabaikan seperti efisiensi, kualitas, kemudahan dipelihara/dikembangkan, dan kecocokan dengan lingkungan yang sebenarnya. Jika klien merasa cocok dengan prototipe yang disajikan dan berkeras terhadap produk tersebut, maka developer harus kerja keras untuk mewujudkan produk tersebut menjadi lebih baik, sesuai kualitas yang seharusnya.

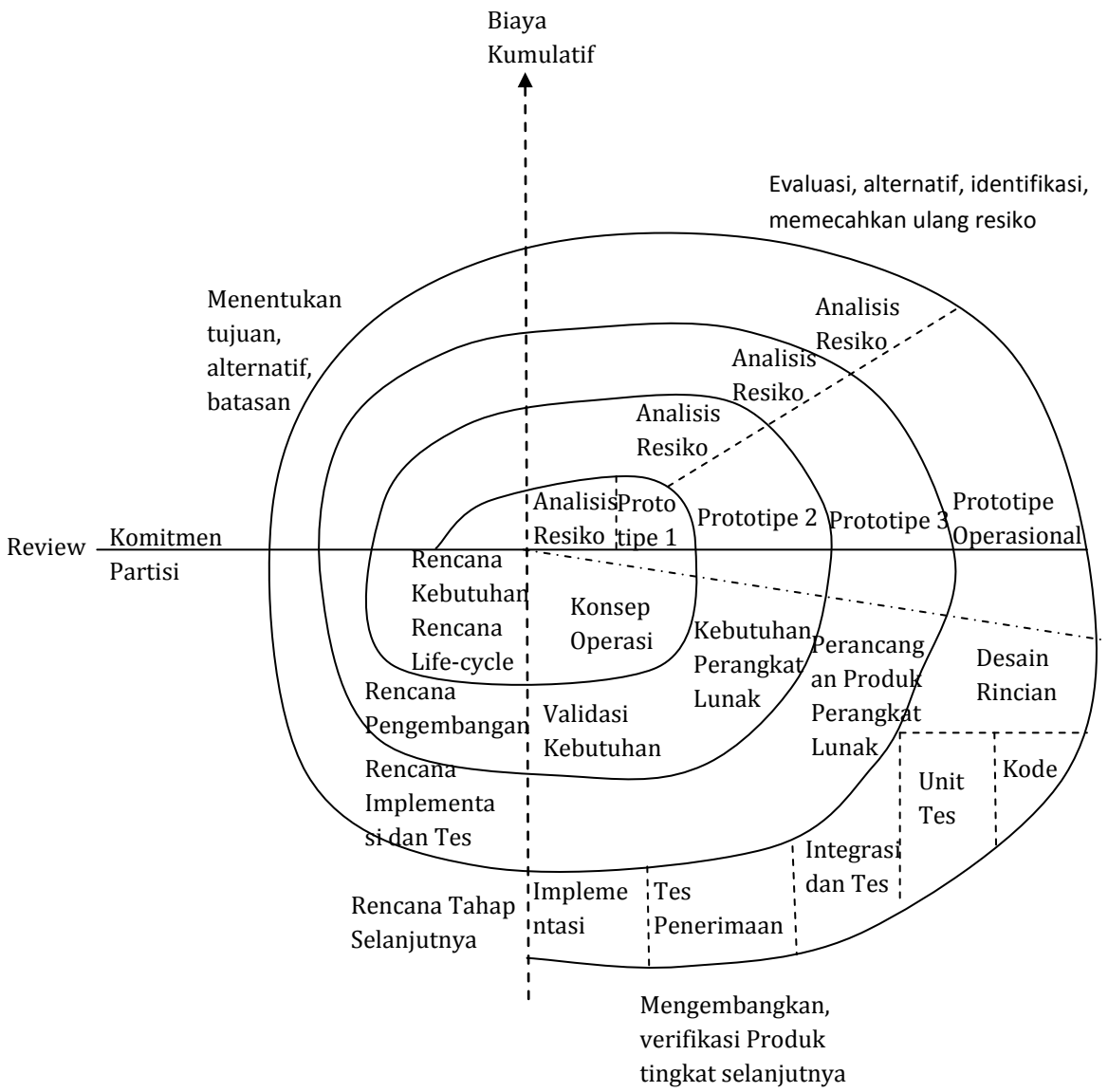
2. Developer biasanya melakukan kompromi dalam beberapa hal karena harus membuat prototipe dalam waktu singkat. Mungkin sistem operasi yang tidak sesuai, bahasa pemrograman yang berbeda, atau algoritma yang lebih sederhana.
3. Agar model ini bisa berjalan dengan baik, perlu disepakati bersama oleh klien dan developer bahwa prototipe yang dibangun merupakan alat untuk mendefinisikan kebutuhan software.

2.4 MODEL SPIRAL

Model ini dikembangkan oleh Boehm pada tahun 1988 didasarkan pada pengalamannya dengan melakukan perbaikan terhadap model *waterfall* yang diaplikasikan pada proyek pemerintah, khususnya perangkat lunak yang besar. Model ini berfokus pembuatan prototipe dan manajemen resiko yang sangat fleksibel bila dibandingkan dengan model *waterfall*. Pengembangan aplikasi dari model ini ada pada pengembangan *TRW-Software Productivity System* (TRW-SPS) yang dijabarkan oleh Boehm.

Dasar-dasar model ini adalah bahwa setiap bagian produk dan setiap tingkatan melibatkan urutan yang sama pada setiap langkah(siklus). Dimulai dari tengah spiral, dapat dilihat bahwa setiap tahapan pengembangan(konsep operasi, kebutuhan perangkat lunak, perancangan produk, rancangan detail dan implementasi) melibatkan satu putaran(siklus) dari spiral. Dimensi radikal pada gambar menunjukkan biaya kumulatif yang terdapat pada penjelasan langkah. Dimensi yang bersudut merepresentasikan kemajuan yang terjadi dalam penyelesaian setiap putaran dari spiral.

Setiap bagian dari spiral adalah untuk identifikasi tujuan dari bagian produk yang diteliti. Disini, alternatif berarti implementasi dari bagian tersebut dan batasan mendesak pada aplikasi alternatif. Analisis resiko dan pendekatan yang dikendalikan resiko menjadi karakteristik utama dari model spiral daripada pendekatan yang dikendalikan dokumen dari model *Waterfall.* Gambar 2.4 menunjukkan model spiral Boehm.



Gambar 2.4. Model Spiral dalam proses Perangkat Lunak

a. Kelebihan model Spiral :

1. Dapat disesuaikan agar perangkat lunak bisa dipakai selama hidup perangkat lunak komputer.
2. Lebih cocok untuk pengembangan sistem dan perangkat lunak skala besar
3. Pengembang dan pemakai dapat lebih mudah memahami dan bereaksi terhadap resiko setiap tingkat evolusi karena perangkat lunak terus bekerja selama proses .
4. Menggunakan prototipe sebagai mekanisme pengurangan resiko dan pada setiap keadaan di dalam evolusi produk.
5. Tetap mengikuti langkah-langkah dalam siklus kehidupan klasik dan memasukkannya ke dalam kerangka kerja iteratif .
6. Membutuhkan pertimbangan langsung terhadp resiko teknis sehingga mengurangiresiko sebelum menjadi permaslahan yang serius.

b. Kelemahan model Spiral :

1. Sulit untuk menyakinkan pelanggan bahwa pendekatan evolusioner ini bisa dikontrol.
2. Memerlukan penaksiran resiko yang masuk akal dan akan menjadi masalah yang serius jika resiko mayor tidak ditemukan dan diatur.
3. Butuh waktu lama untuk menerapkan paradigma ini menuju kepastian yang absolut

2.5 MODEL PROSES PENGEMBANGAN ITERATIF

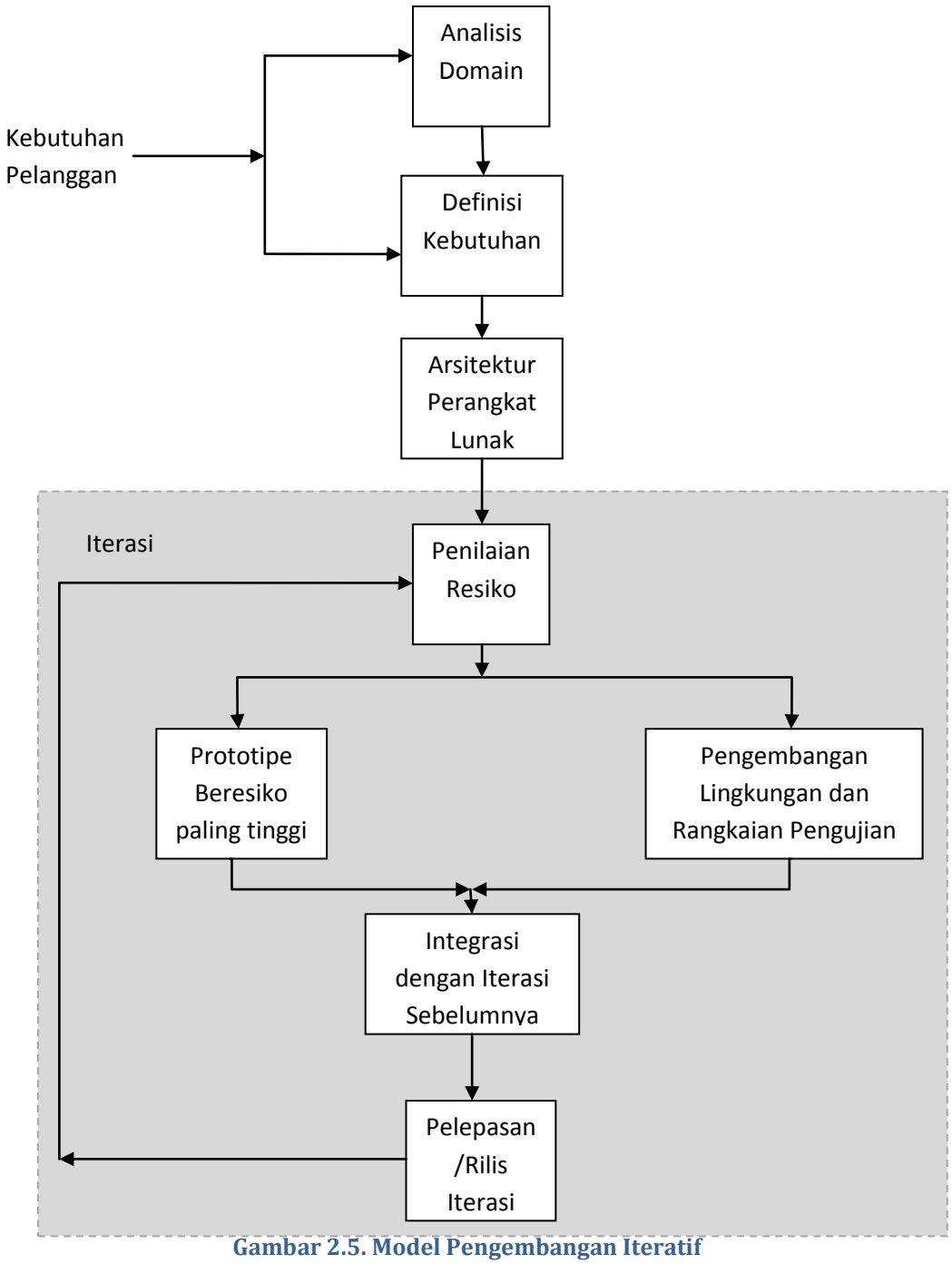
Model Proses pengembangan iteratif atau dikenal *Iterative Development Process* (IDP) diperkenalkan oleh Basili dan Turner 1975. Dengan melakukan analisis pada masing-masing produk, perancangan dan kebutuhan yang dimodifikasi secara terus-menerus melalui serangkaian iterasi untuk mempersiapkan sebuah sistem kepada pengguna yang melakukan perubahan kebutuhan secara bertahap bagi pelanggan dengan rancangan yang telah dikembangkan berdasarkan *feedback* yang diberikan dan proses pembelajaran.

IDP menggabungkan pembuatan prototipe (*prototyping*) dengan kekuatan model *waterfall* klasik. Metode-metode lainnya seperti analisis domain dan resiko juga dapat dimasukkan ke dalam IDP. Model ini dapat disamakan dengan spiral terutama pada pembuatan prototipe dan manajemen resiko. Model Spiral dapat disebut dengan IDP khusus.

Model IDP juga menyediakan sebuah kerangka kerja untuk kebanyakan sistem modern, metode rekayasa perangkat lunak dan tehnik-tehnik seperti penggunaan kembali (*reuse*) , pengembangan berorientasi objek (*object-oriented development*) dan pembuatan prototipe yang cepat (*rapid prototyping*).

Gambar 2.5. menyajikan contoh model pengembangan iteratif, sementara contoh implementasi yang terdiri dari 8 langkah utamanya adalah sebagai berikut :

1. Analisis domain
2. Definisi kebutuhan
3. Arsitektur perangkat lunak
4. Penilaian resiko
5. Prototipe
6. Pengembangan lingkungan dan rangkaian pengujian
7. Integrasi dengan iterasi yang sebelumnya
8. Pelepasan iterasi



Gambar 2.5. Model Pengembangan Iteratif

Pada gambar 2.5. diatas proses iterasi melibatkan lima langkah terakhir. Analisis domain, definisi kebutuhan, dan arsitektur perangkat lunak adalah langkah-langkah pra-iterasi yang serupa dengan langkah-langkah yang ada pada model *waterfall.*

Selama kelima langkah iterasi dilakukan, aktivitas yang akan berlangsung adalah sebagai berikut :

1. Menganalisis atau meninjau kebutuhan-kebutuhan sistem
2. Merancang atau merevisi solusi yang terbaik untuk memenuhi kebutuhan-kebutuhan tersebut,

3. Mengenali resiko-resiko tertinggi bagi proyek dan mendahulukannya. Mengurangi risiko prioritas yang paling tinggi melalui prototipe, meninggalkan risiko yang lebih rendah untuk iterasi bagian berikutnya,
4. Menentukan dan menjadwalkan atau meninjau ulang beberapa iterasi selanjutnya
5. Mengembangkan lingkungan untuk pengujian iterasi yang sesuai dan mendukung lingkungan tes,
6. Menerapkan bagian perancangan yang sedikit dibutuhkan untuk memenuhi iterasi yang sedang berlangsung,
7. Menyatukan perangkat lunak dalam lingkungan pengujian dan melakukan pengujian secara regresi(mundur)
8. Memperbarui dokumen untuk diluncurkan bersamaan dengan iterasi dan
9. Melepaskan/meluncurkan iterasi

Pendukung proses iteratif adalah pendekatan tim-tim kecil yang masing-masing timnya memiliki tanggung jawab penuh terhadap fungsi khusus sistem. Masing-masing tim mempunyai proyek, fungsi kualitas dan pemuasan pelanggan masing-masing dan memegang tanggungjawab penuh. Tim *cross-functional* juga memberikan dukungan dan layanan untuk membuat tim subsistem berhasil dan membantu memutuskan masalah-masalah *cross-subsystem.*

2.6 MODEL PENGEMBANGAN BERORIENTASI OBJEK

Pendekatan berorientasi objek (*object-oriented*) untuk perancangan dan pemrograman diperkenalkan pada tahun 1980-an. Pendekatan ini mewakili suatu pergeseran paradigma utama dalam pengembangan perangkat lunak. Berbeda dengan pemrograman tradisional yang memisahkan data dan kontrol, pemrograman berorientasi objek dilakukan berdasarkan objek, dengan sekumpulan daya yang ditetapkan dan sekumpulan-sekumpulan operasi yang dapat dilaksanakan dalam data tersebut.

Pada awal penyebaran teknologi *Object-Oriented* kebanyakan literatur *object-oriented* membahas mengenai metode analisis dan perancangan. Informasi tentang proses pengembangan *Object-Oriented* juga sangat sedikit. Branson dan Herness mengusulkan suatu proses pengembangan OO untuk proyek berskala besar yang berfokus pada metodologi dengan delapan langkah yang didukung oleh sebuah mekanisme untuk *tracking,* inspeksi, sekumpulan teknologi dan aturan-aturan untuk pembuatan prototipe dan pengujian.

Proses pengembangan yang terdiri dari delapan langkah tersebut dibagi dalam tiga tahap logis yakni :

1. Tahap analisis

   Tahapan ini berpusat pada perolehan dan penyajian kebutuhan pelanggan dengan cara yang singkat untuk menggambarkan sebuah sistem penting yang mewakiliki kebutuhan-kebutuhan pengguna yang kurang diperhatikan dengan platform implementasi yang telah dikembangkan.

2. Tahap Perancangan

Melibatkan perubahan sistem penting sehingga sistem dapat diterapkan pada sekumpulan perangkat keras dan perangkat lunak yang diberikan. Kelas-kelas digabung dan diintegrasikan ke dalam hierarki kelas pengembangan. Tujuan sistesis kelas adalah mengoptimalkan penggunaan kembali (*reuse*) dan menciptakan kelas yang data dipergunakan kembali (*reusebale*)

3. Tahap Implementasi

Mengambil kelas-kelas yang ditetapkan untuk penyelesaian

Kedelapan langkah dari tahapan logis tersebut adalah sebagai berikut :

1. Model Sistem Utama

Sistem utama menggambarkan aspek-aspek yang dibutuhkan sistem untuk mencapai tujuannya dengan memperhatikan lingkungan perangkat keras dan perangkat lunak target. Model ini terdiri dari data dan aktifitas utama.

2. Memperoleh calon kelas utama

Langkah ini menggunakan sebuah teknik yang dikenal dengan *carving* (mengukir) untuk mengidentifikasi metode-metode dan calon-calon kelas utama dari model utama seluruh sistem. *Data-flow diagram* yang lengkap bersama dukungan spesifikasi proses dan masukan kamus data adalah dasar untuk kelas dan pemilihan metode.

3. Membatasi model utama

Model utama diubah untuk bekerja dalam batas lingkungan implementasi target. Aktivitas dan data utama dialokasikan ke berbagai jenis prosesor dan media penyimpanan data.

4. Memperoleh kelas-kelas tambahan

Metode-metode dan kelas-kelas tambahan dikhususkan untuk lingkungan implementasi yang diseleksi berdasarkan aktivitas-aktivitas yang ditambahkan saat membatasi model penting.

5. Mengintegrasikan kelas-kelas

Calon kelas-kelas utama dan tambahan diintegrasikan dan diorganisasikan ke dalam suatu hierarki. Atribut dan operasi digali untuk menghasilkan beberapa superkelas dan subkelas.

6. Menentukan antarmuka

Antarmuka (*interfaces*), deklarasi tipe obyek dan definisi kelas dibuat berdasarkan penggabungan kelas yang didokumentasikan.

7. Menyelesaikan rancangan

Pada langkah ini, rancangan modul implementasi diselesaikan. Modul implementasi meliputi beberapa metode yang masing-masing memberikan sebuah fungsi kohesif tunggal, logika interaksi sistem.

8. Menerapkan solusi

Implementasi dari kelas-kelas telah dikodekan dan unit telah diuji.

Tahap analisis pada langkah diatas adalah langkah a dan b, sementara perancangan terdiri dari langkah c sampai f dan tahap implementasi langkah g dan h

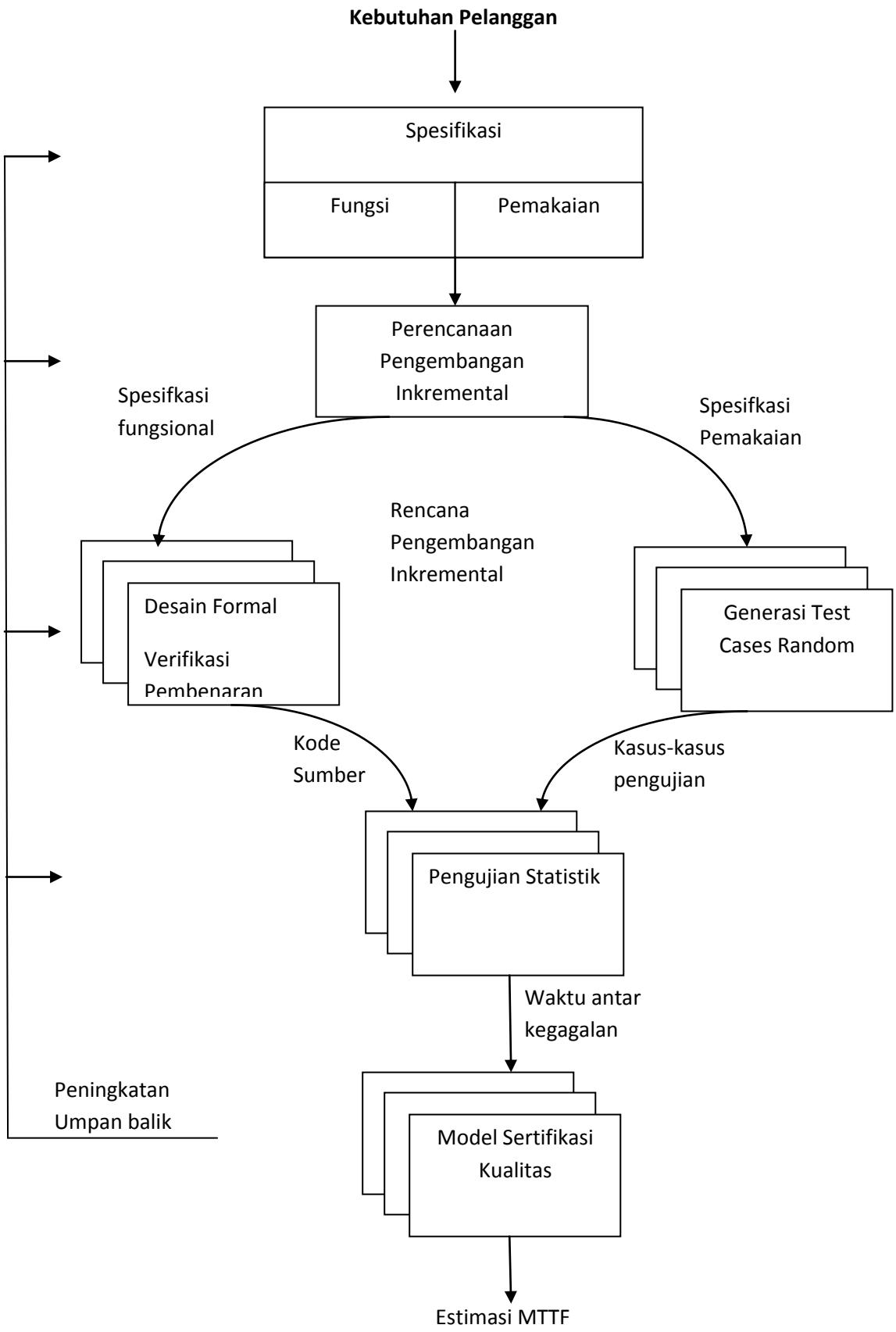
Branson dan Herness(1993) menyatakan bahwa arsitektur proses pengembangan berorientasi objek juga harus memasukkan elemen-elemen seperti dapat digunakan kembali (*reuse*), CASE tool, integrasi, pengembangan dan pengujian serta manajemen proyek.

Dalam aplikasi-aplikasi bisnis, metodologi-metodologi pengembangan aplikasi yang menggunakan bahasa pemrograman terstruktur DFD, ERD kurang mampu beradaptasi dengan kebutuhan dan harapan *user.* Maka pada sekitar tahun 1980-1990 para pakar mulai bekerja dengan bahasa pemrograman yang berorientasi objek seperti C++ dan Java, sehingga diperlukan metodologi dan komponen yang sesuai. UML (*Unified Modelling Language*) merupakan metodologi kolaborasi antara metode Booch, OMT (*Object Modelling Technique*) serta OOSE (*Object Oriented Software Engineering*) dan beberapa metode lainnya, yang merupakan sebuah metode yang paling sering digunakan saat ini untuk mengadaptasi maraknya penggunaan bahasa "pemrograman berorientasi objek".

2.7 METODOLOGI CLEAN ROOM

Proses Rekayasa Perangkat Lunak dengan metodologi *Cleanroom* adalah proses pengembangan yang dikhususkan untuk menghasilkan perangkat lunak dengan tingkat kehandalan yang diakui. Proses *Cleanroom* secara orisinil dikembangkan ileh Harlan Mill dan beberapa koleganya termasuk Alan Hevner di IBM. Fokus dari proses *Cleanroom* adalah pencegahan kerusakan, ketimbang menghilangkan cacat/kerusakan*.* Nama *Cleanroom* dipilih untuk membangkitkan istilah *cleanroom* dalam industri elektronik untuk mencegah pengenalan kerusakan selama pembuatan semikonduktor. Proses *cleanroom* pertama kali tampil digunakan pada pertengahann hingga akhir tahun 80an.

Manajemen *Cleanroom* didasarkan pada pembangunan inkremental dan sertifikasi dari sebuah garis inkremen fungsi pengguna (*user-function*). Gambar 2.6 menunjukkan implementasi penuh dari proses *Cleanroom.*



Gambar 2.6. Proses Clean Room

Proses *cleanroom* menitikberatkan pentingnya tim pengembang untuk mempunyai kontrol intelektual selama proyek berlangsung. Dasar-dasar dari proses tersebut adalah bukti kenyataan rancangan dan kode serta sertifikasi kualitas formal melalui pengujian statistik.

Proses *Cleanroom* mengumumkan bahwa pengujian secara statistik dapat menggantikan cakupan dan bagian pengujian. Dalam *cleanroom ,* semua pengujian didasarkan pada pengantisipasian terhadap cara penggunaan pelanggan. *Test case* dirancangkan untuk menguji fungsi-fungsi yang paling banyak digunakan. Dengan *test case* kesalahan yang dapat mengakibatkan kerusakan pada penggunaan akan dapat ditemukan terlebih dahulu. Dalam hal pengukuran, kualitas perangkat lunak dijamin dengan *Mean Time To Failure*(MTTF).

Proses *Cleanroom* mewakili satu dari pendekatan-pendekatan formal dalam pengembangan perangkat lunak yang telah dimulai untuk melihat aplikasi pada industri. Pendekatan formal lainnya adalah Vienna Development Method(VDM) dan Notasi Z(Smith dan Wood, 1989; Wing, 1990). Z dan VDM telah digunakan pertama kali oleh para pengembang di Inggris dan Eropa, sedangkan proyek *cleanroom* kebanyakan ke Amerika Serikat.

Sejak proyek rintisan pada tahun 1987 dan 1988, sejumlah proyek mulai diselesaikan menggunakan proses *cleanroom.* Seperti dilaporkan oleh Linger (1993), rata-rata tingkat kerusakan pada eksekusi awal adalah 2.9% dari kerusakan per seribu baris kode (KLOC) yang secara signifikan lebih baik ari rata-rata industri.

Kebanyakan pemakaian *cleanroom* sejauh ini hanya dibatas pada proyek-proyek kecil. Seperti metode formal lainnya, pertanyaan tentang peningkatan kemampuan pada proyek skala besar dan kebutuhan pelatihan matematika juga dipertanyakan oleh banyak pengembang dan manajer proyek. Pertanyaan apakah statistik pengujian dapat sepenuhnya menggantikan batasan pengujian dan alur pengujian? menjadi sebuah pertanyaan utama banyak pengembang. Terutama ketika suatu sistem itu komplek atau sistem itu adalah sebuah *common-purpose system* dengan profil cara pemakaian pelanggan tertentu yang juga dipertanyakan. Maka tidak mengejutkan jika beberapa proyek dengan proses *cleanroom* tidak sejalan dengan metode tradisional seperti *unit test.* Hausler dan Trammel (1993) mengusulkan sebuah pendekatan implementasi bertahap dalam memperlancar penerimaan atas *cleanroom.* Kerangka kerja implementasi bertahap terdiri dari tiga tahapan, yakni :

1. Implementasi pendahuluan, yang melibatkan penerapan prinsip-prinsip *cleanroom* tanpa formalitas penuh dari metodologi.
2. Implementasi penuh, yang melibatkan penggunaan lengkap dari metode-metode formal *cleanroom*(seperti gambar 2.6)
3. Implementasi lanjutan, yang mengoptimalkan proses untuk lingkungan lokal semisal pemodelan markov dan analisis dari penggunaan sistem

2.8 PROSES FRAMEWORK MATURITY DAN STANDAR KUALITAS

Dalam sub-bab ini akan dibahas kerangka kerja untuk menilai kematangan proses dari suatu organisasi atau proyek. Pembahasan mencakup SEI, Penilaian Malcolm Baldrige dan ISO 9000.

2.8.1 PROSES SEI *CAPABILITY MATURITY MODEL* (SEI-CMM)

Software Engineering Institute di Universitas Carnegie Mellon mengembangan proses *Capability Maturity Model (CMM)* yang merupakan suatu standar kerangka kerja pengembangan perangkat lunak. CMM meliputi lima tingkatan proses kematangan (Humphrey 1989), yakni

1. Tingkat 1 : Initial Process

   Mayoritas organisasi perangkat lunak berada pada tingkat ini, banyak organisasi hampir tidak pernah meninggalkan Tingkat 1, karakteristik pada tingkatan proses ini adalah semrawut(*chaotic*) - belum terjadwalnya kegiatan, biaya serta pencapaian kualitas yang tidak dapat diprediksi karena tidak adanya kontrol manajemen formal atas pengembangan perangkat lunak. Dalam Tingkatan proses ini dibutuhkan tindakan antara lain perencanaan mulai dari penjadwalan, perkiraan dan besarnya biaya, pengukuran kinerja, pengontrolan kinerja, komitmen manajemen dan *quality assurance*

2. Tingkat 2 : Repeatable Process

   Karakteristik Tingkat ini adalah intuitif – dimana biaya dan kualitas sangat bervariasi,pengendalian penjadwalan yang sudah masuk akal. Prosedur dan metode proses masih bersifat informal dan berjalan terpisah.

   Pada level ini dibutuhkan tindakan untuk pengembangan standar dan definisi proses, membuat sumber daya proses, mengembangkan metode (kebutuhan, desain, inspeksi dan tes). Atau *key process area* yang digunakan pada *maturity level* 2 adalah :

   a. Manajemen Kebutuhan
   b. Perencanaan proyek perangkat lunak
   c. Manajemen subkontrak perangkat lunak
   d. Jaminan kualitas perangkat lunak
   e. Manajemen konfigurasi perangkat lunak

3. Tingkat 3 : Defined

   Karakteristik dari tingkatan ini adalah Kualitatif – Penjadwalan dan biaya yang sudah masuk akal, namun kinerja kualitas tidak dapat diprediksi. Sehingga pada tingkatan ini dibutuhkan tindakan berupa pembuatan pengukuran proses dan tujuan kualitas secara kuantitatif, perencanaan dan pengukurannya. KPA yang digunakan pada *maturity level 3* disini adalah :

   a. Peningkatan proses organisasi
   b. Definisi proses organisasi
   c. Program pelatihan
   d. Manajemen perangkat lunak yang terintegrasi

e. Rekayasa produk perangkat lunak
f. Koordinasi antar kelompok
g. Review peer

4. Tingkat 4 : Managed

Karakteristik dari tingkatan ini adalah Kuantitatif – pengendalian statistik yang masuk akal terhadap kualitas produk. Sementara tindakan yang harus dilakukan pada tingkatan ini adalah perencanaan dan pelacakan produktivitas kuantitatif, lingkungan proses yang diinstrumentasikan, investasi teknologi yang ekonomis. KPA pada tingkatan ini adalah :

5. Analisis dan pengukuran proses
6. Manajemen Kualitas
7. Tingkat 5 : Optimizing

Karakteristik dari tingkatan ini adalah dasar kuantitatif bagi investasi modal yang berkelanjutan dalam proses otomasi dan peningkatan. Tindakan yang dapat dilakukan adalah tekanan yang berkelanjutan pada pengukuran proses dan metode proses untuk pencegahan kesalahan (*error*). KPA pada elevel ini adalah :

a. Pencegahan Error
b. Inovasi Teknologi
c. Manajemen proses perubahan

SEI-CMM telah digunakan oleh banyak lembaga pemerintah dan organisasi perangkat lunak. Metodologi penilaian bersandarpada daftar pertanyaan (85 item pada versi 1 dan 124 item pada versi 1.1) dengan jawaban ya atau tanpa jawaban. Pertanyaan khusus ditunjuk sebagai kunci dan 80% dari semua pertanyaan untuk tingkat tersebut harus dijawab ya.

Tingkat-tingkat kematangan tersebut ditampilkan secara hierarkis. Tingkat 2 harus dicapai sebelum perhitungan tingkat 3 atau yang lebih tinggi diterima. Dan seterusnya ke tingkat 4. Matrik perangkat lunak merupaka karakteristik utama dari kematangan tingkat 4. Sedangkan untuk tingkat 5, elemen dari pencegahan kerusakan merupakan kuncinya.

Penilaian kematangan SEI diselenggarakan pada banyak proyek yang dilakukan oleh SEI atau oleh organisasi itu sendiri dalam bentuk *self-assessment.* Pada april 1996, berdasarkan pada penilaian terhadap 477 organisasi oleh SEI, 68,8% berada pada tingkat 1; 18% berada pada tingkat 2; 11,3% berada pada tingkat 3; 1,5% berada pada tingkat 4; dan hanya 0,4% yang berada pada tingkat 5(Humphrey,2000). Pada maret 2000, berdasarkan pada penilaian terbaru dari 870 organisasi sejak tahun 1995, distribusi persentase meningkat menjadi untuk tingkat 1 39,3%, tingkat 2 36,3%, tingkat 3 17,8%, tingkat 4 4,8% dan tingkat 4 sebanyak 1,8%(Humphrey, 2000).

SEI-CMM diterapkan pada tingkatan proyek atau organisasi. Pada tingkatan perorangan dan tim, Humphrey mengembangkan *Personal Software Process* (PSP) dan *Team Software Process* (TSP). PSP menunjukkan kepada perekayasa perangkat lunak bagaimana cara untuk merencanakan dan menjejaki pekerjaannya, praktik yang konsisten dan baik yang mendorong ke arah pengembangan perangkat lunak berkualitas tinggi. Manajemen waktu, praktik Rekayasa

Perangkat Lunak yang baik, penjajakan data, dan analisis di tingkat perorangan berada di antara area fokus PSP. TSP dibangun pada PSP dan menunjukkan bagaimana menerapkan disiplin rekayasa yang mencakup seluruh tugas tim perangkat lunak.

PSP dan TSP dipandang sebagai versi perorangan dan tim dari CMM secara berturut-turut. Awal tahun 1990-an sejumlah model CMM dikembangkan untuk disiplin yang berbeda. CMMI telah dikembangkan dengan pengintegrasian praktik dari empat CMM, yaitu untuk rekayasa perangkat lunak, rekayasa sistem dan *integrated product and process development* (IPPD). CMMI mempunyai dua penyajian, yaitu penyajian yang dijadwalkan dan penyajian yang berlanjut.

CMMI adalah suatu pendekatan perbaikan proses yang memberikan unsur-unsur penting proses efektif bagi organisasi. Praktik-praktik terbaik CMMI dipublikasikan dalam dokumen-dokumen yang disebut model, yang masing-masing ditujukan untuk berbagai bidang yang berbeda. Terdapat dua bidang minat yang dicakup oleh model CMMI, yakni pengembangan(*development*) dan akuisisi(*acquisition*). Versi terkini CMMI adalah versi 1.2 dengan dua model yang tersedia, yakni CMMI-DEV (CMMI *for development*) yang dirilis agustus 2006 dan ditujukan untuk proses pengembangan produk dan jasa, serta CMMI-ACQ(CMM *for Acquisition*) yang dirilis pada November 2007 dan ditujukan untuk manajemen rantai suplai, akuisisi, serta proses *outsourcing* di pemerintah maupun industri.

Tabel 2.1. Key Process Area CMM

No	Area Proses (KPA)	Singkatan	Kategori	Tingkat
1	*Causal Analysis and Resolution*	CAR	Pendukung	5
2	*Configuration Management*	CM	Pendukung	2
3	*Decision Analysis and Resolution*	DAR	Pendukung	3
4	*Integrated Project Management*	IPM	Manajemen Proyek	3
5	*Measurement and Analysis*	MA	Pendukung	2
6	*Organizational Innovation and Deployment*	IOD	Manajemen Proses	5
7	*Organizational Process Definition*	OPD	Manajemen Proses	3
8	*Organizational Process Focus*	OPF	Manajemen Proses	3
9	*Organizational Process Performance*	OPP	Manajemen Proses	4
10	*Organizational Training*	OT	Manajemen Proses	3
11	*Process and Product Quality Assurance*	PPQA	Pendukung	2
12	*Product Integration*	PI	Teknis	3
13	*Project Monitoring and Control*	PMC	Manajemen Proyek	2
14	*Project Planning*	PP	Manajemen Proyek	2
15	*Quantitative Project Management*	QPM	Manajemen Proyek	4
16	*Requirements Development*	RP	Teknis	3
17	*Requirements Management*	REQM	Teknis	2
18	*Risk Management*	RSKM	Manajemen Proyek	3
19	*Supllier Agreement Management*	SAM	Manajemen Proyek	2
20	*Technical Solution*	TS	Teknis	3

21	*Validation*	VAL	Teknis	3
22	*Verification*	VER	Teknis	3

2.8.2 PENILAIAN MALCOLM BALDRIGE

Malcolm Baldrige National Quality Award (MBNQA) adalah penghargaan kualitas yang paling bergengsi di Amerika Serikat yang didirikan pada tahun 1988 oleh Departemen Perdagangan Amerika Serikat, penghargaan ini diberikan setiap tahunnya kepada perusahaan untuk mengenali dan mengetahui perusahaan yang berprestasi serta memiliki manajemen yang berkualitas. Kriteria pengujian dibagi menjadi 7 kategori yakni :

1. Kepemimpinan
2. Analisis dan informasi
3. Perencanaan kualitas strategi
4. Pemanfaatan sumber daya manusia
5. Jaminan kualitas produk dan jasa
6. Hasil-hasil kualitas
7. Kepuasan pelanggan

Sistem untuk memberikan penilaian item yang terdiri dari 28 item pengujian didasarkan pada tiga dimensi evaluasi yakni pendekatan (*approach*), penyebaran (*deployment*) dan hasil (*result*). Ada 1000 poin yang tersedia dalam kriteria penghargaan.

Untuk menjadi pemenang MBNQA, ada empat elemen dasar yang harus jelas dari pengukuran yaitu :

1. Pengarah, Kepemimpinan dairi tim manajemen Eksekutif senior.
2. Sistem, sekumpulan proses dirancang dengan baik untuk memenuhi pencapaian kebutuhan dan kualitas perusahaan
3. Ukuran kemampuan, hasil dari pengukuran kualitas suatu perusahaan
4. Tujuan, tujuan dasar dari proses yang berkualitas adalah pemberian peningkatan nilai secara terus menerus kepada pelanggan

Pada tahun 1998, 7 kategori MBNQA diorganisasikan ulang menjadi kepemimpinan, perencanaan strategi, fokus konsumen dan pasar, informasi dan analisis, fokus SDM, Manajemen proses dan hasil bisnis. Tidak seperti SEI yang berpusat pada organisasi perangkat lunak, proyek dan proses, MBNQA mencakup bidang yang lebih luas antara lain keseluruhan manajemen kualitas, terkecuali industri dan tentu saja MBNQA mencakup tiga kategori besar yakni manufaktur, layanan, dan usaha kecil.

2.8.3 ISO 9000

ISO 9000 adalah sekumpulan standar dan petunjuk untuk sistem manajemen jaminan kualitas, ISO 9000 menyajikan bagian yang lain dari standar kualitas. ISO 9000 dibuat oleh *International Organizational for Standardization.*

Banyak perusahaan komunitas Eropa terdaftar memiliki ISO 9000, untuk menempatkan produk mereka pada pasar Eropa yang lebih baik, banyak perusahaan Amerika Serikat bekerja keras agar memiliki ISO, untuk memiliki ISO, pemeriksaan resmi pada 20 elemen bagian harus

bernilai positif. Petunjuk untuk aplikasi 20 elemen dijelaskan pada ISO 9000-3. 20 elemen tersebut adalah sebagai berikut :

1. Tanggung jawab manajemen
2. Sistem Kualitas
3. Review kontrak
4. Pemeriksaan perancangan
5. Pemeriksaan dokumen
6. Pembelian
7. Pembeli-penyalur produk
8. Identifikasi dan penelusuran produk
9. Pemeriksaan proses
10. Pemeriksaan dan pengujian
11. Peralatan pemeriksaan, pengukuran dan pengujian
12. Status pemeriksaan dan pengujian
13. Pengawasan produk gagal
14. Tindakan perbaikan
15. Penanganan, penyimpanan, pengemasan dan pengiriman
16. Pencatatan kualitas
17. Pemeriksaan kualitas internal
18. Pelatihan
19. Pelayanan dan
20. Teknik-teknik statistik

Banyak perusahaan mengejar ISO 9000 dan banyak yang gagal pada pemeriksaan pertama dengan tingkat kegagalan hingga 70%. Tingkat kegagalan biasanya disebabkan oleh standar yang kompleks, birokrasi, kesalahan, kurang mengenal persyaratan.

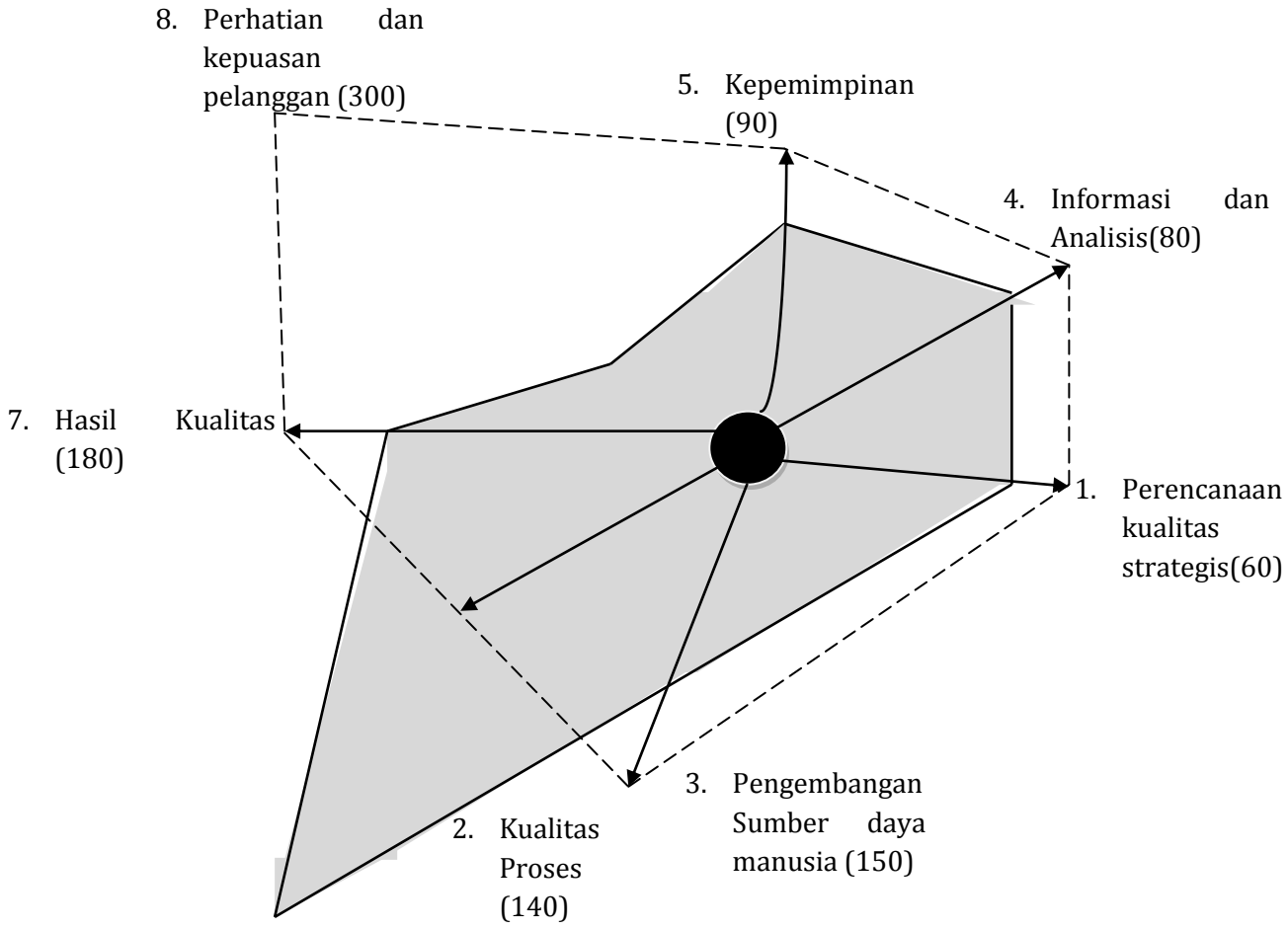
Dari titik paling awal perangkat lunak, tindakan perbaikan dan pengontrolan dokumen adalah bagian yang tidak dapat disesuaikan lagi. Dengan pengontrolan dokumen, ISO 9000 mempunyai persyaratan yang kuat sbb:

1. Cukup memadai untuk mencapai tujuan
2. Pemilik harus dapat diidentifikasikan
3. Disetujui sebelum dipublikasikan
4. Penyebarannya harus diawasi
5. Versinya diidentifikasikan.
6. Nomor halaman
7. Indikasi Total Halaman
8. Yang tidak dipakai lagi dibuang

Kriteria MBNQA dan ISO 9000 dapat saling mengimbangi satu sama lain. Ketika Malcolm Baldrige berpusat pada proses dan hasil, pemeriksaan ISO 9000 berpusat pada kualitas istem manajemen dan pengawasan proses. Secara sederhana ISO 9000 dapat diringkas dengan pernyataan : "Katakan apa yang anda lakukan, lakukan apa yang anda katakan, dan buktikan ". Walaupun ISO 900 tidak memeriksa hasil dan kepuasan pelanggan seperti halnya MBNQA, keduanya saling melengkapi. Gambar 2.7 memberikan gambar perbandingan sistem penilaian ISO 9000 dan Baldrige. Untuk sistem Baldrige panjang anak panah pada tiap kategori

merupakan bagian dari nilai maksimum untuk tiap kategori. Untuk ISO 9000 panjang anak panah didasarkan pada tingginya perhatian dari pengalaman pemeriksaan IBM Rochester ISO 9000, awal pemeriksaan pada tahun 1992 dan tahun-tahun berikutnya.

Jika tingginya tingkatan pada ISO 9000 digabungkan dengan kuatnya disiplin Baldrige, hasil dari sistem akan menghasilkan cakupan yang luas dan penetrasi yang dalam.



Gambar 2.7. Penilaian Malcolm Baldrige dan ISO 9000.

Sinergi Baldrige dan ISO 900 muncul dari hal-hal sebagai berikut :

1. Kebutuhan dokumentasi formal ISO 900
2. Kebutuhan validasi formal ISO memberikan kemudahan kelengkapan dan penyelesaian
3. Tekanan ISO pada tindakan perbaikan memberikan kontribusi oada program peningkatan berkala perusahaan
4. Proses pemeriksaan memberikan tambahan perhatian pada area pemeriksaan

Komite teknis ISO yang bertanggung jawab atas standar kualitas telah mengambil alih proyek besar untuk memperbaharui standar dan membuatnya lebih *user-friendly* . ISO 9001:2000 mencakup perubahan besar pada standarisasi sejak dipublikasikan. Beberapa perubahan utamanya mencakup hal-hal sebagai berikut :

1. Penggunaan pendekatan proses dan struktur baru untuk standar yang dibuat berdasarkan model proses yang menimbang semua pekerjaan pada waktu input dan output
2. Perubahan tekanan prosedur penyediaan dokumen untuk menjelaskan sistem sampai pengembangan dan pengaturan efektivitas proses,
3. Tekanan yang lebih besar pada peran manajemen atas
4. Meningkatkan tekanan pada pelanggam, termasuk keperluan pemahaman, persyaratan pertemuan dan pengukuran kepuasan pelanggan
5. Tekanan pada pengaturan objektivitas terukur dan pengukuran produk dan kinerja proses
6. Pengenalan kebutuhan untuk analisis dan penggunaan data untuk menciptakan peluang peningkatan.
7. Formalisasi konsep peningkatan berkala pada sistem manajemen kualitas,
8. Penggunaan kata-kata yang mudah dimengerti untuk semua sektot produk, tidak hanya perangkat keras,
9. Kelengkapan ketentuan aplikasi untuk mengadopsi ISO 9001:2000 pada semua ukuran dan jenis organisasi pada semua sektor dan pasar.

Dari perubahan ini, kriteria ISO 9000 tampak menyerupai kriteria MBNQA ketika menjaga perhatian pada peningkatan proses.